Helping Eyes: A Coached, Question-Driven Camera Reading Assistant for Visually Impaired Users

Neeraj P Rao, Parveen, Misha
RV University, Bengaluru, India

Manjul Krishna Gupta, Ashwini Kumar Mathur
RV University, Bengaluru, India

***Abstract*—For a visually impaired person, the small print on a medicine strip or a food packet is still hard to get at. Today's reading aids cost a lot, often need special hardware, cope badly with hand-held photos, and read the whole label aloud when the user wants one line. We built *Helping Eyes*, a browser-based assistant served from an ordinary Mac. It finds text in the live camera view and talks the user into position, cleans up the photo with classical computer vision (OCR-seeded GrabCut, homography flattening, dewarping, skew and shadow correction, plus a hands-free book mode), and then answers spoken questions from the captured text alone. A hosted model (Gemini Flash-Lite) or a local one (Qwen 2.5 3B) does the answering and flags questions the text cannot answer. Expiry dates are checked in plain code, and the web is searched only when the user agrees. On 164 real phone photos the app reads 90.5 % of key lines, against 59.2 % for Tesseract. Gemini answers 80.0 % of 355 factual questions correctly and Qwen 64.5 %. When the photo does not contain the answer, both models say so 93.9 % of the time. Doing the expiry check in code took Qwen from 1 correct verdict out of 22 to 22 out of 22.**

***Index Terms*—assistive technology, visual impairment, OCR, document image processing, large language models, grounded question answering, text-to-speech**

I. Introduction

The WHO estimates that at least 2.2 billion people have a vision impairment [1]. A lot of ordinary life still runs on print: the dose on a tablet strip, the expiry date on a packet, a letter, a book. Readers that combine optical character recognition (OCR) with text-to-speech (TTS) have been proposed many times [2]–[4]. Five problems remain open. Commercial devices are expensive and need their own hardware, recognition suffers in poor light and on blurred, shiny, tilted or curved items, and reading out a whole label buries the one fact the user wanted. There is also a problem that only blind users face. They cannot see the viewfinder, so the software has to find the text by itself and tell them how to move.

Helping Eyes is our attempt at this: one web application, served from the user's own Mac and usable on a phone through a secure tunnel. The user holds an item up, follows the spoken hints until the photo is taken, and then asks for what they need, say "read only the dosage" or "when does it expire?".

Contributions. (1) A capture loop for users who cannot see the screen, which turns OCR output and image-quality measurements into spoken hints and takes the photo automatically. (2) A correction pipeline with no trained models, in which GrabCut is started from the OCR boxes. A brightness threshold gave back the photo's own frame for 144 of 145 outlines, whereas the text-seeded method found the item on 150 of 164 photos. (3) A hands-free book mode that notices page turns, finds the spine, orders the text, highlights the spoken word and resumes after the book moves. (4) An answering design that keeps exact jobs (full read-outs, page facts, date arithmetic) away from the language model, gives it an explicit "not in the text" signal and searches the web only with consent. (5) An evaluation on 164 real photos comparing three OCR engines and a hosted against a local 3B model, with an ablation of the expiry check.

II. Related Work

Assistive readers. Earlier OCR-to-speech systems [2]–[4] read out everything they recognise and expect a well-framed picture. The VizWiz work [5], [6] found that photos taken by blind users are often blurred or badly framed, and it introduced visual question answering on such photos. We answer from the OCR text rather than the pixels, so answers can be checked and the photo stays on the Mac.

OCR and document images. Tesseract [7] and PP-OCR [8] are two popular open-source engines. We test both (PP-OCR through RapidOCR) against Apple Vision [9]. The classical toolbox for document photos includes Otsu thresholding [10], CLAHE [11], homography rectification [12], recursive XY-cut for layout [13] and the variance of the Laplacian as a focus measure [14]. Learned dewarping such as DocUNet [15] needs training data and a GPU; we use a simple projection-profile model.

Grounded language models. Language models hallucinate [16], and retrieval-augmented generation [17] counters this by generating from supplied evidence. Our evidence is the captured text. We add an abstention tag, facts computed in code, and a search tool usable only after the user agrees, much like tool-using models [18].

III. System Architecture

Fig. 1 follows one request through the system. The browser handles the camera, microphone and speaker, streams 960-pixel preview frames over a WebSocket and uploads a 1920-pixel photo only at capture. The browser's speech recognition turns requests into text for a command router, and answers come back a sentence at a time, each voiced on the Mac. The FastAPI server makes all decisions. It keeps one session per browser (captured text and the last six exchanges), runs one

OCR job at a time and can require an access code. A hosted model sees only the captured text, never the photo.

IV. Computer Vision Pipeline

The vision code has four parts: A (OCR), B (quality and geometry), C (book mode) and D (live guidance). Apart from the OCR engine it is classical OpenCV and NumPy code with no trained models.

A. Text Recognition (Part A)

We use Apple Vision's `VNRecognizeTextRequest` [9] in accurate mode. For each line it returns the text, a confidence $c \in [0, 1]$, a rotated quadrilateral and word boxes, in about 0.2 s per photo. A live frame counts as text once it has eight characters. At capture, Part B suggests corrected versions only if something looks wrong, and we keep the one with the highest score

$$S = \textstyle\sum_{\ell} c_{\ell}\, |t_{\ell}|, \tag{1}$$

where $|t_{\ell}|$ is the number of characters in line ℓ. The original gets a 5 % bonus, so a correction must clearly help.

B. Image Quality Assessment (Part B)

The checks run on a grey copy of the frame that is 640 pixels wide, and sharpness is measured only inside the text box. For blur we use the variance of the Laplacian [14],

$$B = \operatorname{Var}\big(\nabla^2 I\big), \tag{2}$$

with $\nabla^2 I$ the four-neighbour Laplacian (the four neighbours added up, minus $4I$). A frame is blurred when $B < 60$: sharp print gives strong responses next to zeros, and blur keeps them all small. Exposure comes from the mean grey level μ. The frame is too dark if $\mu < 50$ and too bright if $\mu > 205$ while more than 40 % of pixels are above 245. The standard deviation stands in for contrast, which we call flat below 25. To find glare we mark pixels at 250 or above, clean out specks with a 7×7 morphological opening and label the connected components. If the saturated area is more than 1 % of the frame, we report glare. The centre of the largest blob tells the user which side it is on ("Glare on the right, tilt the page"). Only one hint is spoken at a time: low light first, then glare, blur and over-exposure.

C. Text-Seeded Item Localisation and Flattening (Part B)

Our first outline detector used a brightness threshold, which quietly assumes white paper on a dark table, and it returned the photo's own frame for 144 of its 145 outlines. We now grow the item outward from its own text with GrabCut [19], which separates object from background by colour. The OCR boxes are dilated and joined into clusters, and each line gets the weight

$$w_{\ell} = \frac{A_{\ell}}{1 + (d_{\ell}/0.3)^2}, \tag{3}$$

where A_{ℓ} is the line's area and d_{ℓ} its normalised distance from the centre of the image. The cluster with the largest total weight is taken to be the item's text, which means text on things in the background is ignored. On a 320-pixel copy, the text boxes are sure foreground, the area around them probable foreground and a region up to five times the text block probable background. The border is background unless its Lab colour is within 20 of the colour behind the text, as happens when the item fills the frame. We run four GrabCut iterations, clean the mask with morphology, keep the component that holds most of the text and reduce it to four corners with Douglas–Peucker (falling back to a minimum-area rectangle). We would rather return nothing than a wrong outline, so we give up if the shape covers under 85 % of its convex hull or the four corners leave out over 25 % of it.

If the outline is tilted by more than 2°, or two opposite sides differ in length by more than 4 %, we flatten it with a homography $\mathbf{x}' \sim H\mathbf{x}$ computed from the four corners [12]. The inverse H^{-1} takes the OCR boxes back into camera coordinates so they can be drawn.

D. Curvature, Skew and Tone Correction (Part B)

Dewarping. On an open book the lines curve down into the spine. We binarise the page and cut it into 16 vertical strips. For each strip we take the row profile of ink, and neighbouring profiles are lined up by finding the lag with the highest cross-correlation. That lag is limited to 0.45 of the line pitch, which we read off the first autocorrelation peak. Starting from the strongest strip, the lags are added up going outwards and fitted with a weighted cubic $s(x)$. The page is then remapped as

$$I'(x, y) = I\big(x,\, y + s(x)\big). \tag{4}$$

The result is kept only if the variance of the page's row profile rises by at least 8 %.

Skew. To level the text we rotate the ink mask M_{θ} and look for the angle where its row profile is sharpest inside a central disc,

$$\theta^{*} = \arg\max_{\theta} \textstyle\sum_{y} \big(r_{\theta}(y{+}1) - r_{\theta}(y)\big)^2, \tag{5}$$

where $r_{\theta}(y) = \sum_{x} M_{\theta}(x, y)$. The search covers $\pm 25°$ in $1°$ steps, followed by $0.1°$ steps around the best angle. We rotate (on a larger canvas, so no corner is cut) only if $|\theta^{*}| \geq 1.5°$ and the score beats the unrotated one by 15 %.

Tone. For poor-quality photos we estimate the brightness of the paper by dilating and strongly blurring the lightness channel, and divide it out to remove shadows. CLAHE [11] with 8×8 tiles and a clip limit of 2.0 then evens out local contrast. An adaptive threshold is kept as a last resort. None of these steps changes the geometry, so the OCR boxes remain correct.

E. Coaching a User Who Cannot See the Screen (Part D)

A server-side state machine gives every preview frame a state and, if needed, a spoken hint. The states are SHOW TEXT, ADJUST, HOLD, COACH, CAPTURING and CAPTURED. We say the text is in view when at least 65 % of its bounding box falls inside a guide box. "Move closer" needs the median line height under 14 pixels at 720p, the character-weighted OCR confidence below 0.8 and the item under 75 % of the frame, so small text that is already read well is left alone. Directional

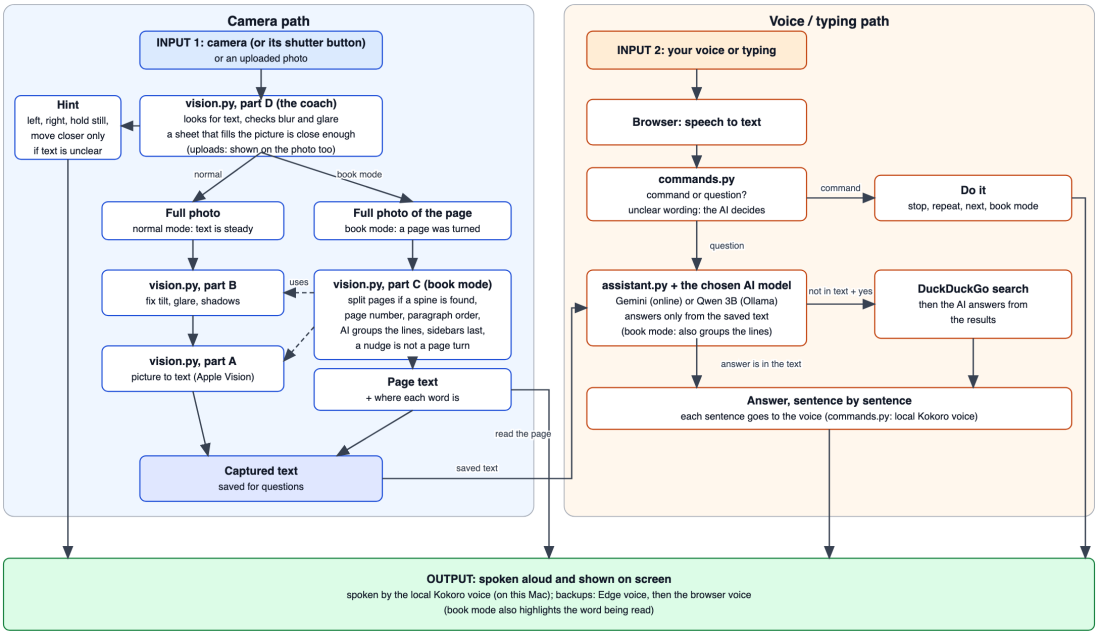


Fig. 1. Life cycle of a request. Left: the camera path through the live coach (part D), photo correction (part B), book analysis (part C) and OCR (part A). Right: the voice/typing path through speech recognition, the command router and the grounded answerer with optional, consented web search. All output is spoken and drawn on screen.

hints ("Move left", "Move up" and so on) are given when the text touches the edge of the guide box and its centre is more than 80 pixels away from the middle. Once the text has been steady and in view for 0.5 s, the photo is taken. If blur or glare does not go away, the user is coached for up to 3 s and the photo is taken regardless, and Part B deals with the problem. To avoid nagging, position hints repeat after 2 s at the earliest, quality hints after 5 s and "Low light" after 30 s. A new capture is armed after 1.5 s without text or on "next".

F. Hands-Free Book Mode (Part C)

When to read. Each frame is shrunk to a blurred 320×180 grey image G_t. Motion is the median of the last five values of $\operatorname{mean}|G_t - G_{t-1}|$, so a single noisy frame cannot restart the timer. A noise floor n is learned from quiet frames as $n \leftarrow 0.95n + 0.05m$. If motion goes above $n + 3$ the page is TURNING, and if it stays below $n + 1.5$ for 0.8 s the page is STEADY. A new page is told apart from a nudged one by layout fingerprints (an adaptive threshold with lines smeared into bands, reduced to 80×45 cells), compared by

$$D(a,b) = \min_{|\delta_x|,|\delta_y| \le 4} \operatorname{mean} |a - \operatorname{shift}(b, \delta_x, \delta_y)|/255, \quad (6)$$

and $D < 0.06$ means the same page has moved. In practice a shift gives about 0.01 and a new page about 0.28.

Where the pages are. The book outline comes from Apple's document segmentation if all corner angles are between 60° and 120° and it holds 90 % of the text; otherwise GrabCut grows it from both pages' text. An outline more than 1.1 times wider than tall is an open spread. The spine is either a valley in the column brightness profile at least 12 % darker than both pages, or the smooth, text-free strip (by local standard deviation) closest to the middle. Either way, both sides need a real block of text, or else the page just has two columns. With no clue we split at the centre.

What order. Rows at the top and bottom of a page count as header or footer when the gap separating them is larger than $\max(1.8\bar{g}, 0.9\bar{h})$, where $\bar{g}$ is the normal gap between lines and $\bar{h}$ the line height. They give the printed page number ("47", "- 47 -", "Page 47" or roman numerals) and the running title, which are announced rather than read as body text. The body itself is ordered with recursive XY-cut [13]. An empty vertical band wider than $1.2\bar{h}$ separates two columns, and the left one is read first. Failing that, a horizontal gap larger than $\max(1.8\bar{g}, \bar{g} + 0.6\bar{h})$ separates a heading from the text below it. An indented line begins a new paragraph, and words split by a hyphen at the end of a line are joined again.

Following the voice. Every character keeps its word's box, mapped back to the camera frame point by point after rotation, homography or dewarping. The browser works out which word is being spoken from how far the audio has played and highlights it (Fig. 2). If the book moves, we line up the new text with the old using Ratcliff–Obershelp matching [20]. When runs of three or more matching words cover 40 % of the shorter text and the page numbers agree, reading resumes from the word it had reached.

V. Language and Speech Layer

A. Request Routing

Regular expressions catch most requests before any model is involved: stop, repeat, new item, mode changes, read everything and common search phrasings. A bare "yes" or "no" is only taken as an answer while a lookup offer is waiting. If

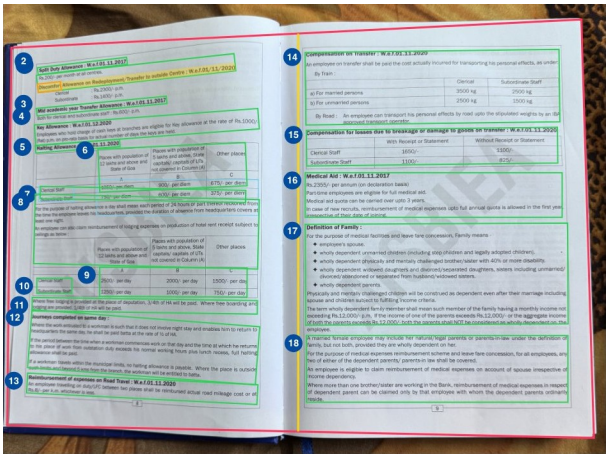


Fig. 2. Book mode on an uploaded two-page spread: book border (red), spine (yellow), paragraph blocks (green) numbered in reading order across both pages, and the line being spoken (highlight).

a request mentions the web but fits none of the patterns, the language model classifies it as `SEARCH: <topic>`, `YES`, `NO` or `OTHER`. Anything left over is treated as a question.

B. Grounded Answering

The system prompt contains the captured text, the current date, the date checks and page facts worked out in code, and a set of rules. Answers must be plain spoken sentences. For a full read-out the model replies `READ_ALL` and the app reads the OCR text itself; requested parts are read word for word; anything else is short and taken only from the text. If the text lacks the answer, the reply starts with `[NOT_IN_TEXT]`, even when the model knows the answer from elsewhere. The reply is streamed and split into sentences as it arrives. A sentence ends only where punctuation is followed by a space and a capital letter, so an amount like "Rs. 25.00" stays in one piece. Each sentence is spoken while the model is still writing the next one. The tags are never read aloud, and a fallback pattern catches abstentions that the model phrases without the tag. "What page is this?" skips the model and uses the parsed page numbers. The user can pick, per request, Gemini Flash-Lite [21] via an OpenAI-compatible API or Qwen 2.5 3B Instruct [22] under Ollama, both at temperature 0.2.

C. Deterministic Expiry Checking

Small models are unreliable at date arithmetic, so we parse and compare dates in code. A date that appears right after an expiry label such as "Exp.", "Use by" or "Best before" is trusted. We handle the formats we saw on real packs, for example `15/04/2028`, `04/2028`, `NOV.26` and `APR 2028`. A date with only a month counts as valid until the last day of that month, and the item is expired exactly when that day is before today. The model is given this verdict and only has to put it into words. Many Indian packs print labels and values in separate columns, so OCR often puts the date far from its label. Then every printed date is checked in code. Gemini picks the expiry date knowing it is the latest one (expiry follows manufacture), the local model is simply given the latest date, and a date already past is flagged as possibly the manufacturing date.

D. Consent-Gated Web Lookup

After an abstention the app asks "Should I look it up online?". Only if the user agrees does the model write a query of at most eight words for DuckDuckGo; the captured text is not sent. The answer is based on the first five results, starts with "According to the web", and for health questions ends by telling the user to check with a pharmacist. The app does not offer lookups for expiry dates, batch numbers or prices, because those belong to the item in hand.

E. LLM-Assisted Paragraphs with Bounded Corrections

For book pages the model gets the numbered lines with layout hints (extent, position, letter height, blocks from a union–find over neighbouring aligned lines, tags such as `[indent]`) and returns JSON with paragraphs, lines to skip and corrections. We throw the reply away if a line appears twice, if more than $\max(2, n/4)$ lines are skipped, or if it takes longer than 12 s. Lines the model forgot are put back in. A correction is only used if it stays close to the printed line. The word count may change by at most $\max(1, n_w/4)$, the length by at most 30 % plus six characters, and the string similarity must be at least 0.6. This lets the model turn "emplovee" into "employee" but stops it from rewriting the author's sentences.

F. Speech Output and Accessibility

Sentences are spoken by Kokoro-82M [23], an open-weight model built on the StyleTTS 2 design [24], at roughly five times real time on the Mac. If it is missing or fails, the free Edge neural voice is used. Any sentence that neither of them manages is spoken by the browser's own voice, which also takes over for 20 s after three failures in a row. Numbers longer than four digits, such as batch and serial numbers, are read one digit at a time. The page fetches three sentences ahead and sends a second request when one is slow, using whichever reply arrives first [25]. Each request carries a generation number, which lets the server drop audio that nobody will play after the user says "stop". The microphone is on only while the app is quiet, with a 700 ms guard against echo. The page follows WCAG 2.1 [26], with 7:1 text contrast, shortcuts that can be switched off, a mode that hands all speech to a screen reader or braille display, and spoken error messages; axe-core found no issues.

VI. Evaluation

A. Setup

We took 164 photos with a phone. Of these, 148 are pharmacy items (79 cartons, bottles and ampoules, 43 tablet strips and 26 personal-care products) and 16 are other things such as food labels, documents, book pages and screens. They cover good light, glare and dim light on flat, tilted and curved surfaces. Runs were made on 28–29 October 2026 on an Apple M3 Pro MacBook (18 GB) with a fixed evaluation date. For

TABLE I
OCR ON 164 REAL PHOTOS (KEY LINES)

Reader	Key-line acc.	CER	Fields found	Time/photo
App capture (ours)	**90.5 %**	**0.095**	**70.8 %**	0.31 s
Apple Vision alone	90.2 %	0.098	70.3 %	0.18 s
RapidOCR (PP-OCR)	88.4 %	0.116	69.3 %	0.69 s
Tesseract	59.2 %	0.408	25.6 %	0.36 s

every photo we prepared a reference listing the key lines and fields (name, strength, batch, dates, price and maker). Each reference was first drafted by an AI model (Claude, from Anthropic) looking only at the photo, before any OCR output had been seen, and then checked by a member of our team. Factual answers are scored with fixed rules that have their own unit tests. A random sample of free-text answers was judged blind in A/B pairs by Claude, which comes from a different company than both models under test. Confidence intervals (95 %) are bootstrapped over photos [27], and prompts were frozen beforehand.

B. Text Recognition

Table I shows the four readers side by side. Apple Vision is well ahead of Tesseract and a little ahead of RapidOCR, which uses PP-OCR models, and it is also faster. Our corrections add a small improvement on top. The flattened version was chosen for 16 photos and the original for 143. By category, accuracy is 99.9 % on documents, 100 % on screens, 96.9 % on cartons, 86.3 % on food labels, 80.9 % on crumpled foil strips and 80.8 % on curved items. The character error rate of 0.095 is still short of the 0.05 we were aiming for.

C. Image Checks on Real Photos

Sharpness ranged from 248 to 7934 and brightness from 66 to 167, so neither ever produced a hint. More surprising to us, the glare check never fired at all. The biggest pure-white blob in any photo was 0.12 % of the frame. Shine on foil, glossy cartons and screens turns out to be bright without actually saturating. The text-seeded outline was found on 150 of 164 photos; the rest were shelves of several products or crumpled strips, where we return nothing on purpose. Slant had a median of 3° and a maximum of 26°.

D. Grounded Answers

Both models got the same 708 questions on 46 photos (30 pharmacy photos sampled by group and all 16 others), asked through the app's own code (Table II). On the 355 factual questions, Gemini is right 15.5 percentage points more often than Qwen (95 % CI 10.8–20.8). Gemini is the only one right on 65 of them and Qwen the only one right on 10. Most of Qwen's mistakes are abstentions on values that are in fact printed, often one row away from their label. Its false-abstention rate is 26.0 %, compared with 12.4 % for Gemini. Part of the error comes from the OCR. Where the OCR had garbled a value, Gemini still recovered it 27.9 % of the time, and Qwen 7.0 %. Neither model invents much: at most 1.7 % of answers contain a number that is not in the text.

TABLE II
HOSTED VS. LOCAL MODEL ON THE SAME QUESTIONS

	Gemini Flash-Lite	Qwen 2.5 3B
Factual answers correct (n=355)	**80.0 %**	64.5 %
value present in OCR text (n=169)	82.2 %	57.4 %
"Not in text" recall (n=131)	93.9 %	93.9 %
False "not in text" (n=169)	12.4 %	26.0 %
Number absent from text (n=708)	1.7 %	0.6 %
Free text correct (blind sample, n=100)	98 %	81 %
First sentence, median / p90	3.1 / 17.7 s	**0.45 / 1.1 s**

TABLE III
"HAS IT EXPIRED?" ON 22 LABEL TEXTS

Who decides	Gemini	Qwen 3B
Code alone	22/22	
App: code verdict worded by model	22/22	22/22
Model alone, app prompt kept	21/22	5/22
Model alone, neutral prompt	22/22	**1/22**

E. Ablation: Expiry Verdicts in Code

We wrote 22 label texts using the date formats from the real packs. Half are expired, and some fall one day either side of the evaluation date (Table III). Without help from the code, Qwen started all 22 of its answers to the neutral prompt with "The text doesn't say", even though every label had its date printed on it. On 34 real packs with the date printed away from its label, both models first reported no expiry date at all. Once every printed date was checked in code, Qwen gave the right date on 21 and Gemini on 22, and each got the verdict right on 26.

F. Web Lookup Safety and Latency

Of 10 consented lookups, all 8 that returned an answer opened with "According to the web" and all 3 health answers ended with the pharmacist reminder, yet when we read them only 4 were correct. Given a made-up manufacturer's name, the model described other companies as if they had made the item, and in one case it made a dietary claim the results did not support. "When was it manufactured?" also slipped past the filter meant to block item-specific lookups.

Table IV lists latencies measured on the real server over 20 runs. Capture and the local model both stay under our 2 s target for the first spoken word. Gemini's free tier was overloaded during our runs (HTTP 503), so its replies took either about 2 s or several minutes. The live check runs 4.8 times per second, although the server on its own could handle 9.3. We present book mode here as part of the system design, and we do not report an evaluation of it on real books in this paper.

VII. DISCUSSION AND LIMITATIONS

The OCR boxes did more work than we expected. They tell GrabCut where the item is and carry the reading highlight through every geometric correction, so we never needed a trained detector. The expiry results changed how we think about model size: the 3B model went from 1 to 22 correct

TABLE IV
LATENCY ON AN M3 PRO (LOCALHOST)

Operation	Median	p90
Live frame round trip (960 px)	0.11 s	0.14 s
Capture, 1920 px photo → text	0.29 s	0.47 s
Kokoro audio for one sentence	1.30 s	1.33 s
Question → first audio, Qwen 3B	1.37 s	1.89 s
Question → first audio, Gemini (busy tier)	9.80 s	103 s

verdicts once the arithmetic moved into code. For safety-critical facts, what the model is allowed to do mattered more than its size. Real photos also exposed rules that looked fine on paper and did nothing in practice, glare being the clearest case. We plan to look for bright, low-saturation patches over text instead (e.g. HSV $V > 235$, $S < 40$).

The system has clear limits. The server needs macOS for Apple Vision, though RapidOCR could replace it elsewhere. Factual accuracy is below the 90 % we aimed for, and on labels printed in two columns a value can end up paired with the wrong label. Web answers can describe the wrong company. The live check is slower than 10 frames per second. All photos come from our team in one city, their references were AI-drafted and checked by one person, and no user study with visually impaired participants has been run yet. Users should still confirm medical details with a pharmacist.

VIII. CONCLUSION

With Helping Eyes we wanted to show that a useful reading assistant needs no special hardware; a laptop, a browser and a free model key are enough. Coached capture and training-free correction get readable text out of hand-held photos, and the language layer answers only the question asked, with exact facts worked out in code. Next we want to support OCR on non-Mac machines, read label layouts as tables, redesign the glare check, add an image-capable model for scenes without text, and run a study with users. The code and evaluation scripts are public [28].

ACKNOWLEDGMENT

We thank RV University for supporting this work, which was carried out as part of the courses CS3235 (Working with Large Language Models) and CS4231 (Computer Vision).

REFERENCES

[1] World Health Organization, "World report on vision," Geneva: WHO, 2019.

[2] A. Sharma, A. Srivastava, and A. Vashishth, "An assistive reading system for visually impaired using OCR and TTS," *International Journal of Computer Applications*, vol. 95, no. 2, pp. 13–18, 2014.

[3] U. Gawande, N. Rathod, P. Bodkhe, P. Kolhe, H. Amlani, and C. Thaokar, "Novel machine learning based text-to-speech device for visually impaired people," in *Proc. 2nd Int. Conf. Smart Technologies and Systems for Next Generation Computing (ICSTSN)*, Villupuram, India, 2023, pp. 1–5.

[4] D. S R, V. K. Gowda, R. Rai R, S. Kumar S, and V. K P, "Smart reader for blind people," in *Proc. Int. Conf. Advances in Power, Signal, and Information Technology (APSIT)*, Bhubaneswar, India, 2025, pp. 1–4.

[5] J. P. Bigham, C. Jayant, H. Ji, G. Little, A. Miller, R. C. Miller, R. Miller, A. Tatarowicz, B. White, S. White, and T. Yeh, "VizWiz: Nearly real-time answers to visual questions," in *Proc. 23rd ACM Symp. User Interface Software and Technology (UIST)*, 2010, pp. 333–342.

[6] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham, "VizWiz grand challenge: Answering visual questions from blind people," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 3608–3617.

[7] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[8] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[9] Apple Inc., "VNRecognizeTextRequest and VNDetectDocumentSegmentationRequest, Vision framework documentation," https://developer.apple.com/documentation/vision, accessed Nov. 2026.

[10] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[11] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.

[12] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[13] G. Nagy and S. Seth, "Hierarchical representation of optically scanned documents," in *Proc. 7th Int. Conf. Pattern Recognition (ICPR)*, 1984, pp. 347–349.

[14] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proc. 15th Int. Conf. Pattern Recognition (ICPR)*, vol. 3, 2000, pp. 314–317.

[15] K. Ma, Z. Shu, X. Bai, J. Wang, and D. Samaras, "DocUNet: Document image unwarping via a stacked U-Net," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4700–4709.

[16] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[19] C. Rother, V. Kolmogorov, and A. Blake, ""GrabCut": Interactive foreground extraction using iterated graph cuts," *ACM Trans. Graphics*, vol. 23, no. 3, pp. 309–314, 2004.

[20] J. W. Ratcliff and D. E. Metzener, "Pattern matching: The gestalt approach," *Dr. Dobb's Journal*, vol. 13, no. 7, pp. 46–51, 1988.

[21] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[22] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[23] hexgrad, "Kokoro-82M: An open-weight text-to-speech model," https://huggingface.co/hexgrad/Kokoro-82M, 2025.

[24] Y. A. Li, C. Han, V. S. Raghavan, G. Mischler, and N. Mesgarani, "StyleTTS 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[25] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[26] World Wide Web Consortium, "Web content accessibility guidelines (WCAG) 2.1," W3C Recommendation, https://www.w3.org/TR/WCAG21/, 2018.

[27] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[28] N. P. Rao, Parveen, and Misha, "Helping eyes: Source code and evaluation scripts," https://github.com/neerajprao/Helping_Eyes, 2026.